

LANGCHAIN — LECTURE 2

Memory & Multi-Tool Agents

Building a Study Assistant That Remembers You

Hands-on Session • VS Code • Groq (100% Free)

AGENDA

Today's Roadmap

1. Quick Recap of Lecture 1
2. The Problem — Agents Forget
3. What Is a Checkpointer?
4. Multiple Tools in One Agent
5. Build: Personal Study Assistant
6. Multi-Turn Conversation Demo
7. Your Turn: Add a New Tool
8. Common Errors + Recap

WHERE WE LEFT OFF

Quick Recap — Lecture 1

STEP 1

Plain LLM

model.invoke() — just chats, no memory, no tools.

→

STEP 2

Prompt + Chain

prompt | model — reusable instructions.

→

STEP 3

Agent + 1 Tool

create_agent() — could use ONE tool (get_weather).

But we noticed 2 big limitations:

1. The agent could only use ONE tool — real assistants need many.
2. Every time we called agent.invoke(), it had ZERO memory of the last question — like talking to someone with amnesia.

PART 1 — NEW CONCEPTS

Today's Goal

We fix both limitations by building a "Personal Study Assistant" that:



Uses Multiple Tools

Picks the right tool automatically — calculator, study tips, and more.



Remembers You

Recalls your name and earlier messages in the same conversation.



Separates Conversations

Keeps different users' chats completely separate from each other.

The Problem — Agents Have No Memory

By default, every single agent.invoke() call starts completely fresh — as if it's the very first message ever.

You: *"Hi, my name is Ali."*

Agent: *"Hello Ali! How can I help?"*

You (new call): *"What is my name?"*

Agent: *"I don't know your name — you haven't told me."*

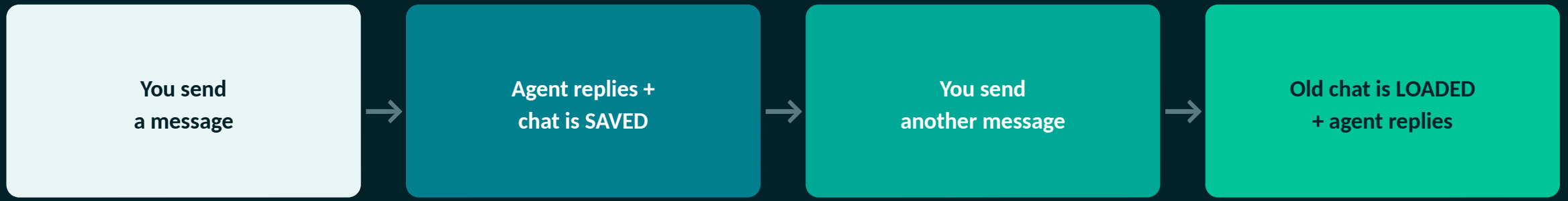
Why? Because each call to model.invoke() or agent.invoke() is stateless — it has NO idea what happened in a previous call, unless we explicitly give it memory.

PART 1 — NEW CONCEPTS

What Is a Checkpointer?



A checkpointer is LangChain's way of saving the conversation after every message, so it can be loaded back the next time you call the agent.



For today's class, we use **InMemorySaver** — it keeps the conversation in your computer's memory (RAM). Simple for learning, but it resets when your program stops. In real production apps, a database-backed checkpointer is used instead.

thread_id — Keeping Conversations Separate



Think of thread_id as a "chat room number"

Every conversation needs a thread_id. As long as you use the SAME thread_id, the agent remembers that specific conversation. A DIFFERENT thread_id means a completely fresh, separate memory.

thread_id: "ali-101"

```
"My name is Ali"
"What is my name?" → "Ali"
```

Agent remembers Ali perfectly inside this thread.

thread_id: "sara-202"

```
"What is my name?" → "I don't know your name yet"
```

Totally different thread — no idea about Ali's conversation.

PART 1 — NEW CONCEPTS

Multiple Tools in One Agent

An agent can have a whole list of tools. It automatically reads each tool's name + docstring and decides which one (if any) fits the user's question.

`calculate_average`

"Marks ka average nikalta hai"

User: "Average of 80, 90, 70?"

`get_study_tip`

"Subject ke liye tip deta hai"

User: "Math ka tip do"

`get_weather`

"Shehar ka mausam batata hai"

User: "Lahore mausam?"

```
tools=[calculate_average, get_study_tip, get_weather]  — just a Python list. The agent handles the "which one?" decision itself.
```

PART 2 — HANDS ON

Practical — What We'll Build



Goal: A Personal Study Assistant

It will remember the student's name across messages, calculate the average of their marks, and give study tips — all through natural conversation.

2 Tools

`calculate_average, get_study_tip`

Memory

`InMemorySaver + thread_id`

Groq Model

`Same llama-3.3-70b as Lecture 1`

Continuing from Lecture 1: same venv, same .env with GROQ_API_KEY. We just add one new package.

SETUP

Setup — One New Package

Open the same project folder from Lecture 1 in VS Code, activate your venv, and run:

```
pip install langgraph
```

Why langgraph?

create_agent() is actually built on top of LangGraph behind the scenes (remember the theory lecture?). The memory tool, InMemorySaver, lives inside the langgraph package, so we install it directly.

Reminder: make sure your terminal shows "(venv)" before installing — otherwise you might install into the wrong Python environment.

HANDS-ON

Step 1 — Define the Tools

Create a new file study_assistant.py :

```
from dotenv import load_dotenv
load_dotenv()
from langchain.tools import tool

@tool
def calculate_average(marks: list[float]) -> float:
    """Marks ki list ka average nikalta hai."""
    return sum(marks) / len(marks)

@tool
def get_study_tip(subject: str) -> str:
    """Kisi subject ke liye ek study tip deta hai."""
    tips = {"math": "Roz thori practice karo.",
            "physics": "Concepts ko real examples se juoro."}
    return tips.get(subject.lower(), "Consistent practice hi sabse acha tip hai!")
```

EXPLANATION

Step 1 — What's New Here?

`@tool`

A "decorator" — one line above a function that turns a normal Python function into something the agent can call.

`list[float]`

A type hint saying this parameter is a list of numbers — helps the agent send the right kind of data.

`tips.get(key, default)`

Python dictionary lookup: if the subject isn't found, it safely returns the default message instead of crashing.

`Two @tool functions`

Both are completely independent — we'll hand BOTH to the agent together in Step 2.

HANDS-ON

Step 2 — Build the Agent With Memory

Add this below your tools:

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

agent = create_agent(
    model="groq:llama-3.3-70b-versatile",
    tools=[calculate_average, get_study_tip],
    system_prompt="Tum ek madadgar study assistant ho.",
    checkpointer=checkpointer,
)
```

Only 2 things changed from Lecture 1: a list of TWO tools instead of one, and the new checkpointer= line.

EXPLANATION

Step 2 — Line by Line

`InMemorySaver()`

Creates a memory store that lives in RAM — this is what actually "remembers" conversations.

`tools=[calculate_average, get_study_tip]`

A Python list — the agent can now choose between BOTH tools, or use neither if not needed.

`checkpointer=checkpointer`

This single line is what turns our old memory-less agent into one that remembers.

`Nothing else changed!`

Same model, same system_prompt style as Lecture 1 — memory is just one added argument.

HANDS-ON

Step 3 — Have a Real Conversation

```
config = {"configurable": {"thread_id": "ali-101"}}

# Turn 1: introduce ourselves
agent.invoke(
    {"messages": [{"role": "user", "content": "Hi, mera naam Ali hai."}]},
    config=config,
)

# Turn 2: test if it remembers
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Mera naam kya hai?"}]},
    config=config,
)
print(result["messages"][-1].content)
```

Notice: the SAME config (same thread_id) is passed to both calls — that is the entire trick.

HANDS-ON RESULT

Step 3 — Run It

Run: `python study_assistant.py`

Expected output:

```
Aapka naam Ali hai.
```

This is the moment. Two completely separate agent.invoke() calls, and the second one correctly recalled information from the first — purely because they shared the same thread_id.

Try this: change "ali-101" to a brand new thread_id like "test-999" and ask "Mera naam kya hai?" again — it will say it doesn't know. That proves memory is scoped per thread_id.

HANDS-ON

Step 4 — Watch It Pick the Right Tool(s)

```
result = agent.invoke(
    {"messages": [{"role": "user", "content":
        "Mere marks hain 80, 90, 70 — average nikaalo, "
        "aur math ke liye ek tip bhi do."}]},
    config=config,
)
print(result["messages"][-1].content)
```

Expected-style output:

```
Aapke marks ka average 80 hai. Math ke liye tip: roz thori practice karo.
```

One single message, but the agent silently called BOTH calculate_average AND get_study_tip, then combined the results into one natural reply.

EXERCISE

Your Turn — Add a Third Tool

Write one more @tool function and add it to the tools list. Try one of these:

`get_grade_letter`

Takes a percentage, returns A/B/C/F.

`count_study_hours`

Adds up a list of daily study hours.

`recommend_break`

Given hours studied, suggests a break length.

```
@tool
def get_grade_letter(percentage: float) -> str:
    """Percentage se grade letter batata hai."""
    if percentage >= 80: return "A"
    elif percentage >= 60: return "B"
    else: return "C"

agent = create_agent(..., tools=[calculate_average, get_study_tip, get_grade_letter], ...)
```

Test it in the same conversation (same thread_id) — the agent should now juggle all three tools correctly.

TROUBLESHOOTING

Common Errors and How to Fix Them

`ModuleNotFoundError: langgraph`	You forgot Setup step. Activate venv, then run: pip install langgraph
`Agent still "forgets"`	Check you passed the SAME config (same thread_id) to every invoke() call.
`Different users see the same memory`	Give each user their own unique thread_id — never reuse one thread_id for two people.
`Wrong tool gets called`	Improve the docstring — make it more specific about exactly when to use that tool.
`TypeError on tool arguments`	Check your type hints (e.g. list[float]) match what you're actually asking the user for.

WRAP UP

Recap of Today

1. By default, agents are stateless — every invoke() call starts fresh
2. A checkpointer (e.g. InMemorySaver) saves and reloads conversation history
3. thread_id scopes memory — same id = same memory, different id = fresh memory
4. tools=[...] can hold many functions — the agent picks the right one(s) itself
5. One message can trigger multiple tool calls, combined into a single natural reply
6. Adding memory to an existing agent takes just ONE new argument: checkpointer=



Your Agent Remembers Now!

Next lecture: giving agents knowledge with RAG (Retrieval Augmented Generation).